

Tìm 3 API public thật viết về một domain quan tâm

- Github REST:
 - Loại API: REST
 - Base URL: <https://api.github.com>
 - Auth method: Github Token gửi qua Header
 - Versioning: Có
 - Resource Identifier: Số nguyên hoặc Slug, Name
- OpenWeather:
 - Loại API: REST
 - Base URL: <https://api.openweathermap.org>
 - Auth method: API Key gửi qua Query Parameter
 - Versioning: Có
 - Resource Identifier: Số nguyên hoặc tọa độ, tên địa danh
- Shopee
 - Loại API: REST
 - Base URL: <https://partner.shopeemobile.com>
 - Auth method: Kết hợp giữa access_token và Chữ ký điện tử động
 - Versioning: Có
 - Resource Identifier: Số nguyên hoặc string quy tắc riêng

So sánh 5 đặc điểm của 1 API tốt

- Contract-first
 - Geewax: Chuẩn hóa các method, bổ sung Field Mask và cờ validateOnly để request rõ ràng hơn
 - Higginbotham: Thiết kế dựa trên nhu cầu thực tế, làm việc cùng nhau và tạo Mock API sớm để kiểm tra
- Abstraction
 - Geewax: API Resource không nhất thiết giống CSDL, đồng thời che giấu những xử lý phức tạp bên trong
 - Higginbotham: Ẩn phần phức tạp của hệ thống, có thể dùng Gateway để tách client khỏi các microservice phía sau
- Reusability
 - Geewax: Thiết kế theo các pattern chuẩn nên các API có thể kết hợp và tái sử dụng dễ dàng

- Higginbotham: Xem API như một sản phẩm để nhiều bên có thể sử dụng lại
- Language-agnostic
 - Geewax: Dùng các kiểu dữ liệu và quy ước chung để API có thể làm việc với nhiều ngôn ngữ
 - Higginbotham: Có thể chọn REST, GraphQL hay gRPC tùy vào nhu cầu và loại client
- Versioned & Observable
 - Geewax: Bổ sung các cách tăng độ tin cậy và quản lý resource như request deduplication, soft delete
 - Higginbotham: Chú trọng quản lý vòng đời API, gồm versioning, deprecation và đảm bảo API vẫn tương thích khi thay đổi

```
from flask import Flask, jsonify, request
app = Flask(__name__)
_next = 1
BOOKS = [
    {
        "id": 1,
        "title": "Clean Code",
        "author": "R. Martin"
    }
]

def find(bid):
    return next((b for b in BOOKS if b["id"] == bid),
None)

@app.route("/books", methods=["GET"])
def list_books():
    # (a)
    limit = int(request.args.get("limit", 100))
    q = request.args.get("q", "").lower()
    _sort = request.args.get("sort")

    items = BOOKS
    if q:
        items = [b for b in BOOKS if (q in
b["author"].lower() or q in b["title"].lower())]

    if _sort:
        items = sorted(items, key=lambda x: x[_sort])
```

```

        return jsonify(items[:limit]), 200

@app.route("/books/<int:bid>", methods=["GET"])
def get_books(bid):
    book = find(bid)
    if not book:
        return jsonify({"error": "not found"}), 404
    return jsonify(book), 200

@app.route("/books", methods=["POST"])
def create_books():
    global _next
    body = request.get_json(silent=True) or {}
    t, a = body.get("title"), body.get("author")
    if not t or not a:
        return jsonify({"error": "need title+author"}),
400

    # (c)
    year = body.get("year")
    if year != None and (not isinstance(year, int) or
year < 1900):
        return jsonify({"error": "year phai la so nguyen
>= 1900"}), 400

    _next+=1
    book = {"id":_next, "title":t, "author":a}
    BOOKS.append(book)
    return jsonify(book), 201,
{"Location":f"/books/{book['id']}"}

@app.route("/books/<int:bid>", methods=["PUT", "DELETE"])
def modify_book(bid):
    book = find(bid)
    if not book:
        return jsonify({"error": "not found"}), 404
    if request.method == "PUT":
        # (c)
        body = request.get_json(silent=True) or {}
        if "year" in body:
            year = body["year"]
            if year != None and (not isinstance(year,
int) or year < 1900):
                return jsonify({"error": "year phai
la so nguyen >= 1900"}), 400
            book.update(body)
            return jsonify(book), 200
        return "", 204

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=True)

```

